Queen Mary University of London
School of Electronic Engineering and Computer Science

Lab 4 – Memory Management

Aims

A. The aim of the first part of the lab exercises is to compile the C code in file **virtual.c** that outputs virtual addresses; then run the non-interactive program, understand what is being output by the program, and what it entails.

B. The aim of the second part is to compile the C code in file **stack.c** which calls a recursive function; then run the program and use a disassembler to analyse the machine code, to understand what is being output by the program (and why).

C. The aim of the third part is to compile the C code in file **paging.c** which simulates x86-64 paging using 4-level page tables; then run the interactive program and analyse how page allocation affects the overhead of the simulated page table tree and how the simulation's memory allocations on the heap affect the number of page faults.

The 3 files C source files mentioned above are provided as "starter code" for this lab; they can be found in QMplus. The same applies to the **Lab4 Answer Sheet**; you should print a copy and take it with you to the lab session.

To complete this lab, it is assumed that you can already do the following things:

1. Run shell commands, in particular:
    a. Compile C code with the `gcc` compiler (e.g., command `gcc -o code code.c`)
    b. Execute machine code on a Ubuntu/MacOSX machine (e.g., command `./code`)
    c. Read machine code with a disassembler (e.g., command `objdump -d code`)
    d. Monitor memory-related statistics of processes (e.g., using commands `ps` or `top`)
2. Understand basics of C code; particularly useful to understand are:
    a. pointer-related operators such as `&x` (address), `*x` (dereference) and `x[i]`
    b. pointer types such as `void*` (generic) and `long*` (signed 64-bit integer pointer)
    c. bit-wise operators `x << y` (left shift) and `x & y` (bitwise AND)
3. Compile/execute/use the interactive simulator from file **paging.c**, to simulate memory usage for a set of allocated pages/frames.

Tip: Referring to your **Lab1** sheet should also help with (at least some of) the above commands.

Part A: Virtual Addresses

The code in file **virtual.c** outputs the virtual address (of the first byte) of various variables in both hexadecimal (the most commonly used for addresses) and decimal notations. The code can be compiled by running from a Unix/Linux shell; please go to the folder with the code and execute the shell command:

```
gcc -o virtual virtual.c
```

The code can then be executed by the following shell command:

```
./virtual
```

Your task is to analyse the C code, to help you do the following:

1. understand the program's outputs;
2. reason about virtual *versus* physical address spaces;
3. answer questions **Q1.1**, **Q1.2**, **Q1.3** in the **Lab 4 Answer Sheet**.

The output will be dependent on the machine, but just to give an example it could look like shown in **Figure A.1**.

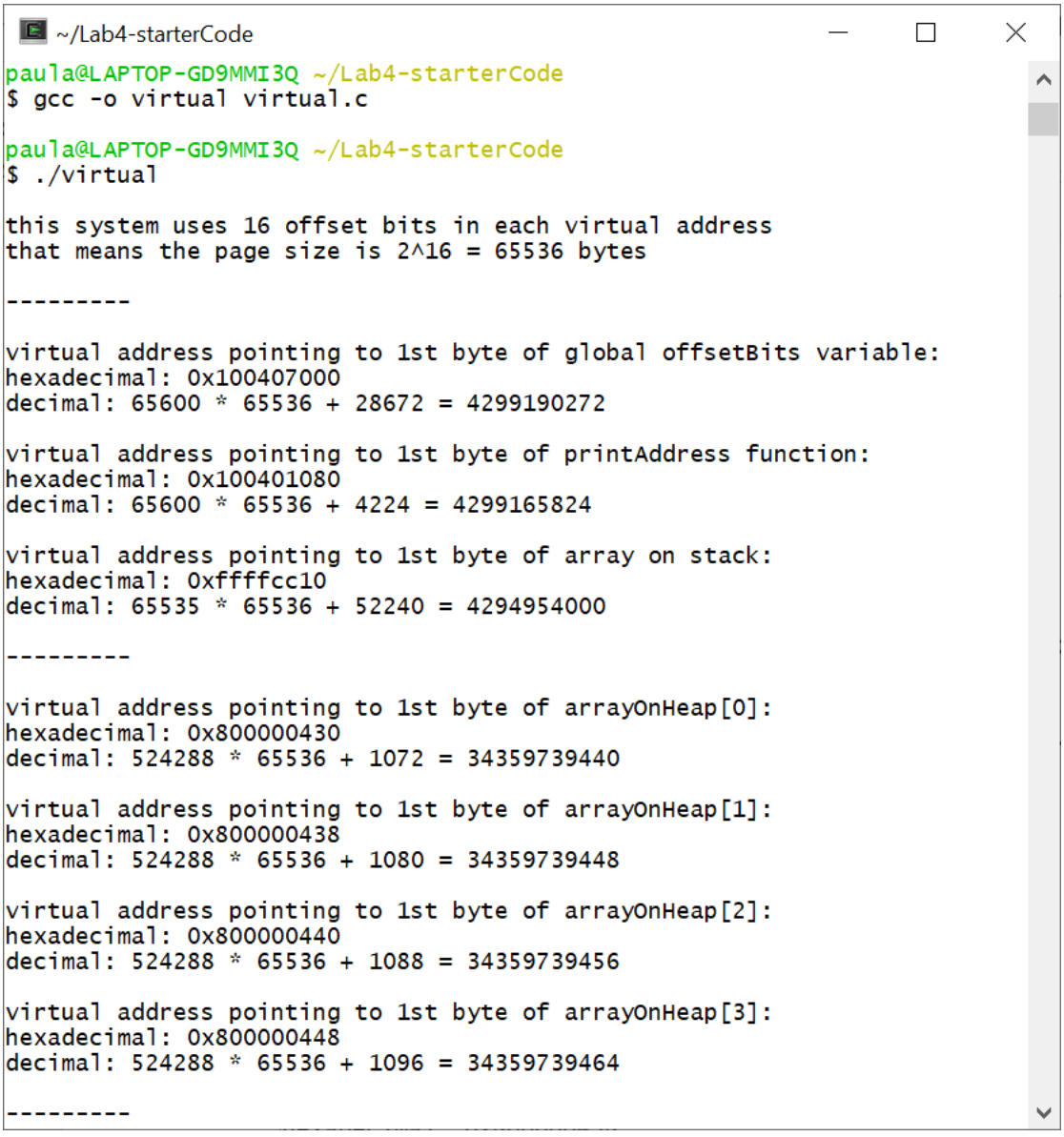


Figure A.1

Part B: Stack

The code in file **stack.c** calls a recursive function. Use a disassembler (suggestions for tools can be found in the code e.g., **`objdump`**) to inspect the executable file. The code can be compiled by running from a Unix/Linux shell; please go to the folder with the code and execute the shell commands:

```
gcc -o stack0 stack.c
gcc -o stack3 stack.c
```

The code can then be executed by the following shell commands:

```
./stack0
./stack3
```

The code can then be disassembled by the following shell commands:

```
objdump –d stack0
objdump –d stack3
```

For MacOSX users only (requiring XCode command line tools), you will need to run the commands:

```
otool -vt stack0
otool -vt stack3
```

Your task is to do the following:

1. use a “disassembler” to understand how function calls work in machine code;
2. analyse the output and link it to the observations about the machine code;
3. answer questions **Q2.1**, **Q2.2**, **Q2.3** in the **Lab 4 Answer Sheet**.

The output will be dependent on the machine, but just to give an example it could look like what is shown in **Figure B.1** (which is just an extract of what you will likely see).

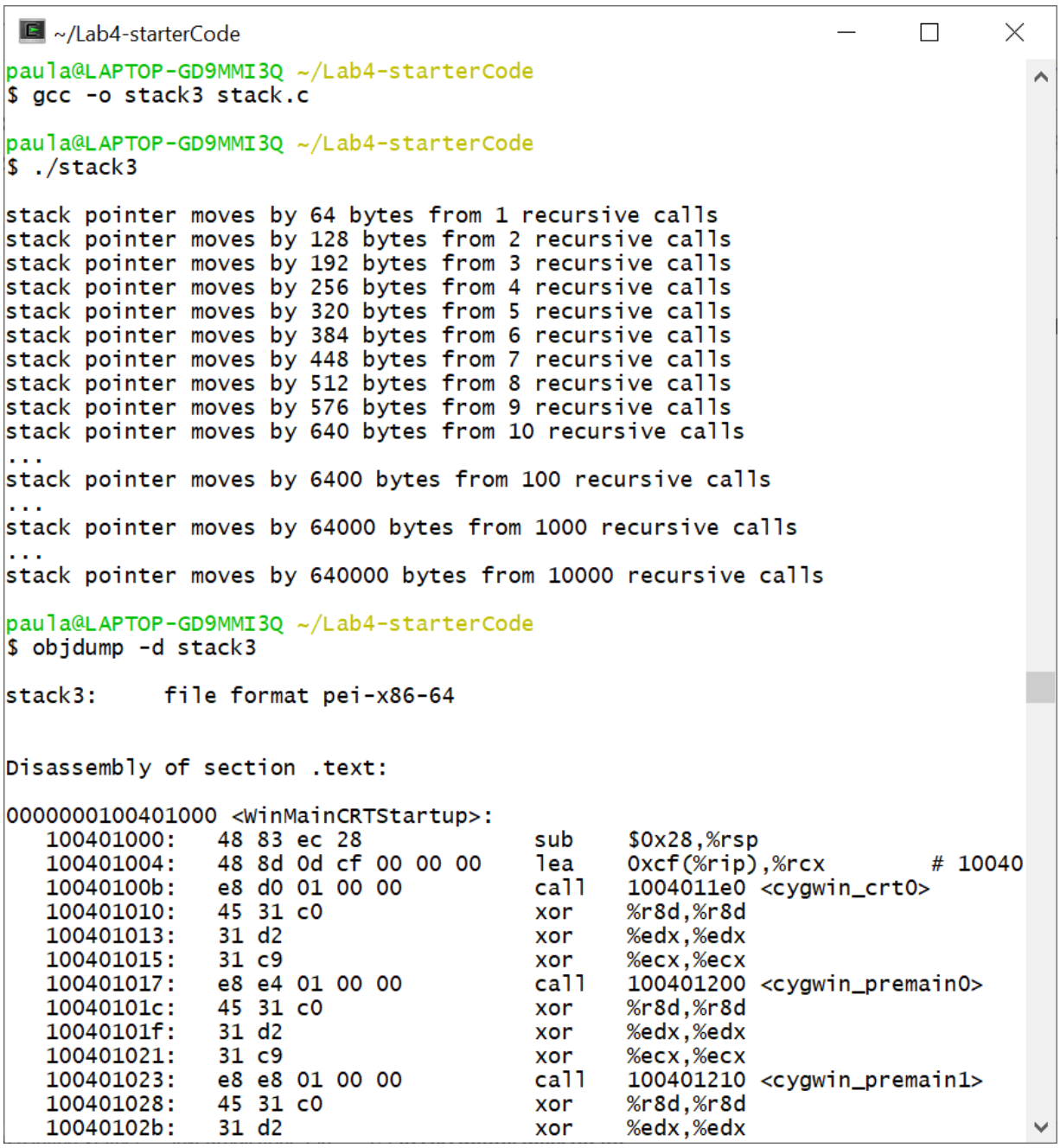

~/Lab4-starterCode
paula@LAPTOP-GD9MMI3Q ~/Lab4-starterCode
$ gcc -o stack3 stack.c
paula@LAPTOP-GD9MMI3Q ~/Lab4-starterCode
$ ./stack3
stack pointer moves by 64 bytes from 1 recursive calls
stack pointer moves by 128 bytes from 2 recursive calls
stack pointer moves by 192 bytes from 3 recursive calls
stack pointer moves by 256 bytes from 4 recursive calls
stack pointer moves by 320 bytes from 5 recursive calls
stack pointer moves by 384 bytes from 6 recursive calls
stack pointer moves by 448 bytes from 7 recursive calls
stack pointer moves by 512 bytes from 8 recursive calls
stack pointer moves by 576 bytes from 9 recursive calls
stack pointer moves by 640 bytes from 10 recursive calls
...
stack pointer moves by 6400 bytes from 100 recursive calls
...
stack pointer moves by 64000 bytes from 1000 recursive calls
...
stack pointer moves by 640000 bytes from 10000 recursive calls
paula@LAPTOP-GD9MMI3Q ~/Lab4-starterCode
$ objdump -d stack3
stack3: file format pei-x86-64
Disassembly of section .text:
0000000100401000 <WinMainCRTStartup>:
100401000: 48 83 ec 28 sub $0x28,%rsp
100401004: 48 8d 0d cf 00 00 00 lea 0xcf(%rip),%rcx # 10040
10040100b: e8 d0 01 00 00 call 1004011e0 <cygwin_crt0>
100401010: 45 31 c0 xor %r8d,%r8d
100401013: 31 d2 xor %edx,%edx
100401015: 31 c9 xor %ecx,%ecx
100401017: e8 e4 01 00 00 call 100401200 <cygwin_premain0>
10040101c: 45 31 c0 xor %r8d,%r8d
10040101f: 31 d2 xor %edx,%edx
100401021: 31 c9 xor %ecx,%ecx
100401023: e8 e8 01 00 00 call 100401210 <cygwin_premain1>
100401028: 45 31 c0 xor %r8d,%r8d
10040102b: 31 d2 xor %edx,%edx

Figure B.1

Part C: Paging using 4-Level Page Tables and Page Faults

The code in file **paging.c** simulates the 4-level page table scheme (AMD) in **Figure C.1**, which has 48-bit virtual addresses (36-bit page number and 12-bit offset; the page number bits are evenly split between the 4 levels into per-level indices); this is very common in x86-64 bit architectures.

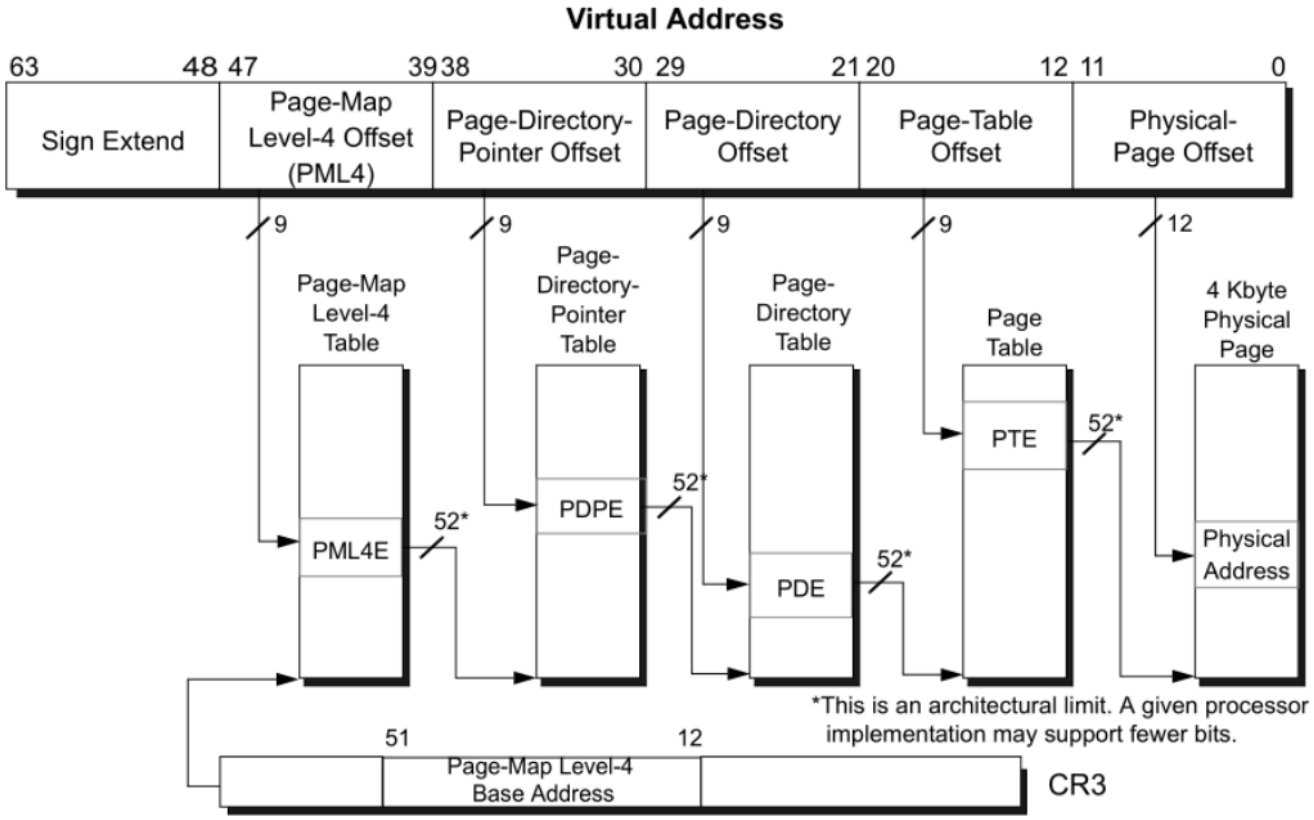


Figure C.1

For the sake of simplicity, we deviate from the above naming scheme (AMD) as indicated in **Table C.1** below:

Intel	AMD	Name throughout Lab
Page-Map Level-4 Table (PML4)		Level-1 Page Table
Page-Directory-Pointers Table (PDPT)	Page-Directory-Pointers (PDP)	Level-2 Page Table
Page-Directory (PD)		Level-3 Page Table
Page-Table (PT)		Level-4 Page Table

Table C.1

The code can be compiled by running from a Unix/Linux shell; please go to the folder with the code and execute the shell command:

```
gcc -o paging paging.c
```

The code can then be executed by the following shell command:

```
./paging
```

Your task is to do the following:

1. monitor number of page faults after allocating more and more page numbers;
2. monitor how much overhead is caused by the 4-level page table scheme;
3. answer questions **Q3.1**, **Q3.2**, **Q3.3** in the **Lab 4 Answer Sheet**.

The program in file **paging.c** has an interactive shell that asks a user input for a page number, or range of page numbers, and then allocates each of those page numbers with a "frame" using the above scheme. After each user input it outputs how much memory is used for the page tables (overhead) and "frames". The **Figure C.2** below shows an example interaction with the paging simulator.

Note: The program runs as a user process and therefore cannot translate virtual to physical addresses directly; instead, it simulates that by translating page numbers to virtual addresses of "frames" that are allocated using a system call (`malloc`). It is only meant to simulate the memory usage involved (which does not require direct knowledge of the physical addresses).

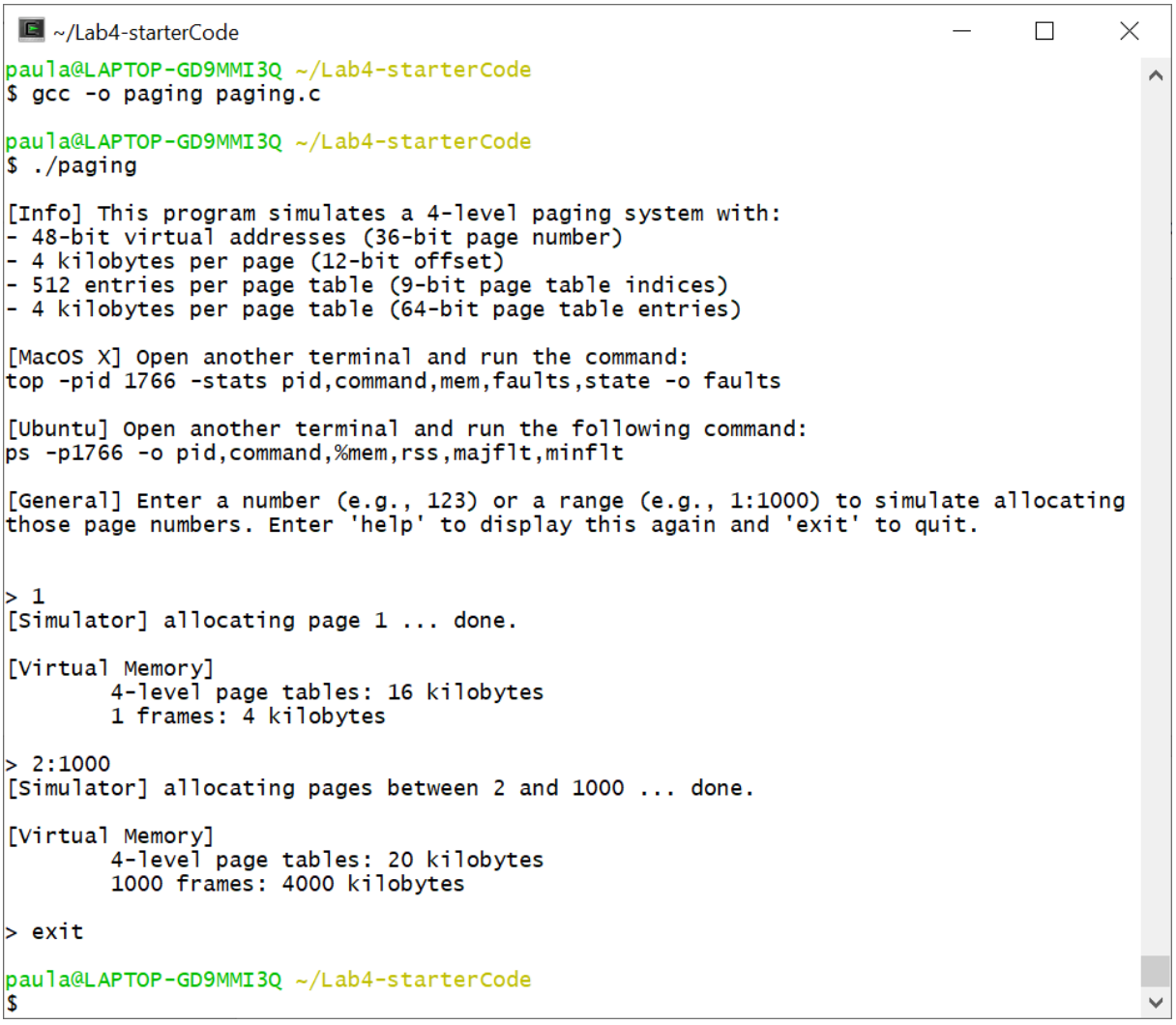


Figure C.2